

Cafe Kubera POS

Detailed documentation of system flowcharts, architecture topologies, database designs, and integrations for self-ordering and Point of Sale administration.

AUTHOR

Antigravity AI Engineer

TARGET VERSION

Production Spec 1.1

GENERATED DATE

July 15, 2026

ARCHITECTURE ENGINE

GraphQL + WebSocket Dual Flow

Welcome to the detailed system documentation for **Cafe Kubera**, an advanced hybrid Point of Sale (POS) and table-ordering platform designed for restaurants and coffee shops.

This document details the complete system architecture, data models, integration layers, and actor-specific workflows. It also provides detailed [Mermaid](#) flowcharts illustrating how components communicate in real time.

1. System Architecture Overview

Cafe Kubera uses a dual-engine architecture designed for high availability and offline resilience. The system can toggle between a cloud-native database flow and a file-based storage flow.

```

graph TD
    %% Frontend Layer
    subgraph Client [Angular Frontend Application]
        Menu[ItemsMenu Component]
        Cart[ItemsCart Component]
        Captain[CaptainPage Component]
        Kitchen[KitchenPage Component]
        Payment[Payment Component]
        Admin[Admin Component]
    end

    %% Sync & Cache Layer
    subgraph Cache [Cache & Real-Time Sync]
        Redis[(Upstash Redis Cache)]
        WS[SockJS WebSocket Server]
    end

    %% Storage & Database Layer
    subgraph Storage [Data & Backup Engines]
        Hasura[(Hasura GraphQL Engine)]
        Dropbox[(Dropbox File Store)]
        QStash[(Upstash QStash Queue)]
    end

    %% External Notifications
    subgraph Notify [External Notifications]
        VercelMail[Vercel Serverless Mailer]
        CallMeBot[CallMeBot WhatsApp]
    end

    %% Connections
    Menu <--> |Read Menu Items| Redis
    Redis <--> |Sync Menu| Hasura
    Cart --> |Create Order| Hasura
    Cart --> |Upload CSV Backup| Dropbox
    Captain <--> |Real-Time Broadcasts| WS
    Kitchen <--> |Real-Time Broadcasts| WS
    Payment --> |Publish JSON Dump| QStash
    QStash --> |Forward DB Updates| Hasura
    Payment --> |Trigger Mail Notification| VercelMail
    Payment --> |Trigger WhatsApp Notification| CallMeBot

```

Key Architectural Components

- **Frontend:** Angular client application using standard components for POS Invoice rendering, Kitchen Order Tickets (KOT), Cart management, Captain actions, and Admin auditing.
 - **Database Flow** (`USE_DATABASE = true`): Relies on a **Hasura GraphQL Engine** mapped to a PostgreSQL database for ordering, payment records, customer profiles, and configurations.
 - **Dropbox Fallback Flow** (`USE_DATABASE = false`): Encodes orders and payment details into CSV format and syncs directly with **Dropbox API v2** paths (e.g., `/orders/approval_waiting_orders/` , `/orders/approved_orders/` , `/orders/paid_orders/`).
 - **Real-time Notifications:** Implemented via a **SockJS WebSocket Server** (`kuber-backup.onrender.com`) to propagate order changes (e.g., waiting, approved, check-out, and paid statuses) instantly to dashboard instances.
 - **Caching & Queueing:** Uses **Upstash Redis** for fast food menu loads, and **Upstash QStash** to queue and forward billing information to Hasura REST endpoints.
 - **Integrations:** Incorporates **Tesseract OCR** for processing payment receipts and an **SMTP Serverless Mailer** on Vercel for sending notifications to administrators and customers.
-

2. Order Status State Machine Flow

Orders navigate through a strict set of states to maintain service accuracy. Below is the transition flowchart showing the lifecycle of an order and the corresponding backend events.

```
stateDiagram-v2
    [*] --> approval_waiting : Item Selection & Order Placed
    note right of approval_waiting
        - DB order saved
        - Dropbox CSV uploaded
        - WebSocket alert sent
    end note

    approval_waiting --> Approved : Waiter / Captain Approves
    note right of Approved
        - Status set to Approved
        - CSV moved to approved_orders
        - Kitchen notified
    end note

    approval_waiting --> Cancelled : Waiter / Captain Rejects

    Approved --> Preparing : Kitchen Staff Ack / print KOT
    Preparing --> Ready : Kitchen Sets Item Completed

    Ready --> checkout : Table Requests Check-Out
    note right of checkout
        - Multiple table orders combined
        - Unique checkout ID assigned
        - Cashier billing triggered
    end note

    checkout --> paid : Cashier Confirms Payment
    note right of paid
        - Transaction written to DB
        - QStash log dispatched
        - Backup CSV archived
        - Loyalty points awarded
        - Confirmation email sent
    end note

    paid --> [*] : Bill Closed
```


3. Database vs. Dropbox Storage Routing Logic

The application features runtime routing using the `USE_DATABASE` toggle. Below is the detailed decision tree showing how data transactions behave in both database and filesystem modes.

flowchart TD

```
Start([Data Operation Triggered]) --> CheckMode{Is USE_DATABASE == true?}
```

%% Database Flow

```
CheckMode -->|Yes| DB_Insert[Interact with Hasura GraphQL API]
```

```
DB_Insert --> DB_Verify{Check Response}
```

```
DB_Verify -->|Success| DB_Finish[Complete Operation & Update UI State]
```

```
DB_Verify -->|Error| DB_Retry[Log Error & Fallback to Local Session Cache]
```

%% Dropbox Flow

```
CheckMode -->|No| CSV_Gen[Convert Entity Objects to CSV string using PapaParse]
```

```
CSV_Gen --> Dropbox_Upload[Upload CSV file to Dropbox API v2 via DropboxService]
```

```
Dropbox_Upload --> Drop_Verify{Check API Response}
```

```
Drop_Verify -->|Success| Drop_Finish[Complete Operation & Broadcast Event via Socket]
```

```
Drop_Verify -->|Error| Drop_Retry[Attempt Dropbox File Update / Retry Endpoint]
```

4. real-time WebSockets & Sound Alert Flow

Instant synchronization between front-of-house waiters, kitchen chefs, and the checkout cashier is handled by SockJS events. Here is how alerts propagate:

flowchart LR

Sender[Action Component] -->|WebSocket Send| SocketServer[kuber-backup WebSocket Server]

SocketServer -->|Broadcast Message| Receivers{Message Type Routing}

Receivers -->|'approval'| Captain[Captain Dashboard]

Captain -->|Play Audio| AudioWait[order_waiting.mp3 / bell.mp3]

Captain -->|Refresh View| ShowWait[Show Pending Order Card]

Receivers -->|'kitchen'| Kitchen[Kitchen Screen]

Kitchen -->|Play Audio| AudioKit[kitchen.mp3]

Kitchen -->|Refresh View| ShowKit[Show Prepared Item Card]

Receivers -->|'payment'| Cashier[Payment Dashboard]

Cashier -->|Play Audio| AudioPay[bell.mp3]

Cashier -->|Refresh View| ShowPay[Show Bill Checkout Card]

5. UPI ID Rotation & Dynamic Payment QR Code Logic

To balance payment volume and remain within daily UPI transaction limits, the checkout system dynamically switches between merchant addresses.

flowchart TD

```
TriggerInvoice[Invoice Requested for Table] --> GetTrigger{GET rest/triggerquery from Hasura}  
GetTrigger --> CheckVal{Is trigger value == 'y'?}
```

%% Route Y

```
CheckVal -->|Yes| UseUPI2[Set UPI ID = KUBERA2_UPI_ID]  
UseUPI2 --> ResetTrigger[POST rest/trigegrupdaten to reset value to 'n']  
ResetTrigger --> GenQR2[Generate dynamic upi://pay link with Total & Bill No]
```

%% Route N

```
CheckVal -->|No| UseUPI1[Set UPI ID = KUBERA_UPI_ID]  
UseUPI1 --> GenQR1[Generate dynamic upi://pay link with Total & Bill No]
```

```
GenQR2 & GenQR1 --> GenDataURL[Use QRCode.toDataURL to encode Link as base64 Image]  
GenDataURL --> EmbedPrint[Embed QR code on Invoice layout & invoke printPage]
```

6. Core Actor Workflows

Workflow A: Customer Self-Ordering & Cart Submission

1. **Menu Load:** `ItemsMenuComponent` checks **Upstash Redis** for cached menu configurations. If it is a cache miss, it pulls from Hasura, caches the result in Redis, and populates the view.
2. **Item Configuration:** Customer configures items (size, customizations, flavors) and adds them to `ItemsCartComponent`.
3. **Loyalty Lookup:** If a customer provides their phone number, the app checks if they have a valid, active membership. If they do, a **10% discount** is automatically applied.
4. **Submission:**
 - Creates a unique order reference ID.
 - Inserts the record into the database with status `approval_waiting`.
 - Uploads a backup CSV of the order to Dropbox `/orders/approval_waiting_orders/`.
 - Sends a WebSocket broadcast containing the string `"approval"` to notify the Captain page.

```
sequenceDiagram
    autonumber
    actor Customer
    participant Menu as ItemsMenu Component
    participant Cart as ItemsCart Component
    participant Redis as Upstash Redis
    participant DB as Hasura GraphQL
    participant WS as WebSocket Server
    participant Drop as Dropbox Storage

    Customer->>Menu: Open URL / Scan QR Code
    Menu->>Redis: Check Cache
    alt Cache Hit
        Redis-->>Menu: Return Menu Data
    else Cache Miss
        Menu->>DB: Query Menu REST API
        DB-->>Menu: Return Menu JSON
        Menu->>Redis: Set Cache (Key: menu_data)
    end
    Customer->>Menu: Select Item, Size & Flavor
    Menu->>Cart: Add to Cart
    Customer->>Cart: View Cart & Input Phone Number
    Cart->>DB: Check Member Status & Fetch Points
    alt Membership Active
        DB-->>Cart: Return Customer Details & Valid Member Card
        Cart->>Cart: Apply 10% Discount
    end
    Customer->>Cart: Click "Submit Order"
    Cart->>Cart: Generate Order Ref ID (Status: approval_waiting)
    Cart->>DB: Mutation: SaveOrderWithItems
    DB-->>Cart: Return Order ID
    Cart->>Drop: Upload CSV to /orders/approval_waiting_orders/
    Cart->>WS: Send Broadcast Message "approval"
    Cart-->>Customer: Show Order Placed (Pending Captain Approval)
```

Workflow B: Captain Order Review, Approval & KOT Broadcast

- Alert Trigger:** The Captain dashboard receives the `"approval"` WebSocket broadcast and plays `order_waiting.mp3` or `bell.mp3`.
- Order Review:** The Captain reviews pending orders in the "Waiting Orders" tab.
- Approval:**
 - Updates the status to `Approved` in Hasura.

- Moves the corresponding CSV file in Dropbox from `/orders/approval_waiting_orders/` to `/orders/approved_orders/`.
- Broadcasts a WebSocket message `"kitchen"`.

```
sequenceDiagram
    autonumber
    actor Captain
    participant CapComponent as CaptainPage Component
    participant WS as WebSocket Server
    participant DB as Hasura GraphQL
    participant Drop as Dropbox Storage

    WS->>CapComponent: Receive Broadcast "approval"
    CapComponent->>CapComponent: Play audio: order_waiting.mp3
    CapComponent->>CapComponent: Display Order Card in "Waiting Orders"
    Captain->>CapComponent: Click "Approve Order"
    alt USE_DATABASE = true
        CapComponent->>DB: Mutation: updateOrderStatus (Status: Approved)
        DB-->>CapComponent: Confirm Update
    else USE_DATABASE = false
        CapComponent->>Drop: Move CSV file to /orders/approved_orders/
    end
    CapComponent->>WS: Send Broadcast Message "kitchen"
    CapComponent->>CapComponent: Move Card to "Approved Orders"
```

Workflow C: Kitchen Queue & KOT Thermal Ticket Printing

1. **Kitchen Alert:** Kitchen dashboard receives `"kitchen"` WebSocket broadcast and triggers `kitchen.mp3`.
2. **Print KOT:** Kitchen staff opens the order details modal, which uses `PosKotComponent` to generate a formatted thermal receipt view (275px wide).
3. **Physical Print:** The print dialog opens to output to a receipt printer.
4. **Preparation:** Kitchen prepares the items.

```
sequenceDiagram
    autonumber
    actor Chef
    participant KitComponent as KitchenPage Component
    participant KOT as PosKot Component
    participant WS as WebSocket Server

    WS->>KitComponent: Receive Broadcast "kitchen"
    KitComponent->>KitComponent: Play audio: kitchen.mp3
    KitComponent->>KitComponent: Load Approved Orders Queue
    Chef->>KitComponent: Open Order details
    KitComponent->>KOT: Initialize printData
    Chef->>KOT: Click "Print KOT"
    KOT->>KOT: Open Print Window & Inject KOT CSS (275px width)
    KOT-->>Chef: Launch Browser Print Dialog
```

Workflow D: Table Checkout & Dynamic UPI QR Code Generation

1. **Checkout Request:** Customer asks for the bill, or the Captain initiates checkout.
2. **Aggregation:** The Captain dashboard combines all active orders for the designated table.
3. **Status Shift:** Updates status of these orders to `checkout` and generates a combined `check_out_id`.
4. **Broadcast:** Sends a `"payment"` WebSocket broadcast.
5. **Bill View:** Cashier accesses the combined bill in `PaymentComponent`.
6. **UPI ID Rotation:** The invoice component queries the trigger database (`glorious-marten-67.hasura.app/api/rest/triggerquery`) to decide which UPI ID to use:
 - If response is `n`, it uses `KUBERA_UPI_ID` (`Q146632442@ybl`).
 - If response is `y`, it uses `KUBERA2_UPI_ID` (`8143119408@ybl`) and invokes `trigegrupdaten` to reset the flag back to `n`.
7. **Dynamic QR Code:** Generates a dynamic QR code based on the selected UPI ID and grand total, embedding it directly on the receipt printed via `PosInvoiceComponent`.

```
sequenceDiagram
    autonumber
    actor Cashier
    participant PayComponent as Payment Component
    participant Invoice as PosInvoice Component
    participant DB as Trigger Hasura API
    participant QR as QRCode Utility

    PayComponent->>PayComponent: Load Checkout Orders grouped by check_out_id
    PayComponent->>Invoice: Select Table Bill for Invoice print
    Invoice->>DB: GET: glorious-marten-67/.../triggerquery
    DB-->>Invoice: Return trigger state ("y" or "n")
    alt Trigger = "y"
        Invoice->>Invoice: Select KUBERA2_UPI_ID
        Invoice->>DB: POST: /trigegrupdaten (reset trigger flag to "n")
    else Trigger = "n"
        Invoice->>Invoice: Select KUBERA_UPI_ID
    end
    Invoice->>QR: Generate upi://pay link with chosen UPI, Shop Name & Grand Total
    QR-->>Invoice: Return base64 QR Image DataURL
    Cashier->>Invoice: Click "Print Invoice"
    Invoice-->>Cashier: Launch Browser Print Dialog (Includes Dynamic QR Code)
```

Workflow E: Cashier Settlement, QStash Backups & Loyalty Awards

1. **Settle Bill:** Cashier enters paid amount, selects payment mode (Cash, Card, Online), and clicks confirm.
2. **Insert Payment:** Creates payment transaction details in Hasura via `insertPaymentDetails`.
3. **Update Status:** Moves all related orders to status `paid`.
4. **QStash Dump:** Posts the payload JSON to **Upstash QStash**, which reliably publishes a dump of the transaction details to Hasura's REST API `/api/rest/orderdump`.
5. **Dropbox Archive:** Uploads the paid CSV details to Dropbox `/orders/paid_orders/` and deletes files from `/orders/checkout_orders/`.
6. **Admin Email Notification:** Generates a CSV containing the receipt details and emails it to `cafe kubera2223@gmail.com` via Vercel.
7. **Loyalty Credit:** Calculates loyalty points gained from the purchase, credits them to the customer profile in Hasura, and sends the customer a notification email showing their new balance.

sequenceDiagram

autonumber

actor Cashier

participant PayComponent as Payment Component

participant DB as Hasura GraphQL

participant QStash as Upstash QStash

participant Drop as Dropbox Storage

participant Mail as Vercel Mail API

Cashier->>PayComponent: Click "Receive Payment" (Set cash/card mode)

PayComponent->>DB: POST: insertPaymentDetails (Amount, Bill No, Mode)

PayComponent->>DB: Mutation: updateOrderStatus (Set order statuses to "paid")

PayComponent->>QStash: POST: Publish Order JSON to queue

QStash->>DB: Forward Order Dump to REST /api/rest/orderdump

PayComponent->>Drop: Upload Paid Order CSV to /orders/paid_orders/

PayComponent->>Drop: Delete Temporary CSVs from /orders/checkout_orders/

PayComponent->>Mail: Send Transaction Summary to shop admin (cafe kubera2223@gmail.com)

alt Loyalty Details Exist

PayComponent->>DB: Credit Loyalty Points & Update History

PayComponent->>Mail: Send Loyalty Points Added Notification to Customer's Email

end

PayComponent-->>Cashier: Clear checkout row & update cashier summary

7. Data Models & Database Schema

The Hasura schema contains the following tables:

1. `kubera_order`

Tracks order metadata, table locations, and totals.

- `id` (Int, Primary Key): Autoincrement order ID (printed on KOTs).
- `order_ref_id` (String): Unique transaction reference string.
- `table_no` (String): Designated dining table number.
- `table_place` (String): Location/zone (e.g., Indoor, Garden, Terrace).
- `order_summary_amount` (Numeric): Base cost of all items combined.
- `order_additional_service_amount` (Numeric): Taxes, service charges, or CGST/SGST.
- `order_total_amount` (Numeric): Grand total.
- `order_status` (String): Current phase (`approval_waiting` , `Approved` , `checkout` , `paid` , `print`).
- `comments` (String): Special preparation requests from customer.
- `employee` (String): Captain/Waiter username who approved/placed the order.
- `customer_number` (String): Mobile number for loyalty updates.
- `check_out_id` (String): Grouping key assigned during table checkout.
- `created_at` (Timestamp): Timestamp when created.

2. `kubera_order_item`

Detailed breakdown of items inside an order.

- `id` (Int, Primary Key)
- `order_id` (Int): Foreign key mapped to `kubera_order.id`.
- `item_name` (String): Name of food/drink (includes flavor/size details).
- `item_quantity` (Int): Ordered quantity.
- `item_cost` (Numeric): Single unit cost.
- `item_description` (String): Custom comments.
- `status` (String): Preparation status (`progress` , `completed`).

3. `kubera_payment_details`

Audit trail of customer settlements.

- `id` (Int, Primary Key)

- `order_id` (String): Mapped order ID(s).
- `payment_mode` (String): Payment channel (`cash` , `online` , `Card`).
- `paid_amount` (Numeric): Amount paid by customer.
- `actual_amount` (Numeric): Calculated actual bill value.
- `bill_no` (String): Invoice number (generated using date strings).
- `created_at` (String): Transaction date (IST format).
- `created_time` (String): Transaction timestamp (IST format).

4. Customer Loyalty & Profile Schema (`kubera_profile_...` tables)

- `customer_details` : ID, email, phone number, name, and password hash.
 - `customer_points` : Available loyalty points, lifetime accumulated points.
 - `customer_member_ship` : Expiry details, purchase date, and membership type.
-

8. Key Integrations & Core Configurations

These settings are defined in [constants.ts](#):

- **USE_DATABASE**: `true` enables Hasura GraphQL reads and writes; `false` falls back to Dropbox files.
- **UPSTASH_REDIS_REST_URL**: Caches compiled menu objects, allowing faster loads and reducing GraphQL database lookups.
- **KUBERA_API_WEB_SOCKET_URL**: SockJS WebSocket URL (`https://kuber-backup.onrender.com/ws`) used to coordinate order state changes in real time.
- **QSTASH_DESTINATION_URL**: Target endpoint (`https://alive-bedbug-11.hasura.app/api/rest/orderdump`) where queued orders are sent.
- **UPI IDs**:
 - Primary: `Q146632442@ybl` (`KUBERA_UPI_ID`)
 - Alternative: `8143119408@ybl` (`KUBERA2_UPI_ID`)
- **WhatsApp alerts**: Integrated with CallMeBot using a telephone endpoint (`KUBERA_WHATSAPP_PHONE = '918143119408'`).